

30-Day Daywise Schedule (content yet to be finalized)

Day	Curated Topic & Sub-topics (To be Finalized, this is for ref.)	First Principles & Value Add
PART I: OPERATING SYSTEMS & UNIX/LINUX FOUNDATIONS (DAYS 1-3)		
Day 1	Operating Systems: Core Execution - Processes vs. Threads - CPU Scheduling mechanisms - Virtual Memory & Paging - Context switching overhead - Thread safety & race conditions	Destroys the 'black box' view of how programs execute, showing how to write thread-safe and resource-efficient code that works in harmony with the OS.
Day 2	UNIX & Linux: CLI Fundamentals - Linux Filesystem Hierarchy (FHS) - Terminal navigation & core shell - Standard streams (stdin, stdout, stderr) - Redirection and Piping () - Text filtering (Grep, Awk, Sed)	Transitions students from GUI interfaces to command-line execution, unlocking terminal productivity and laying the groundwork for backend debugging.
Day 3	Linux Systems & Admin Automation - User permissions (chmod, chown) - Process management (ps, kill, top) - Environment variables & configuration - Writing automation bash scripts - System automation via Cron	Empowers interns to automate environment setups, manage system permissions, and handle remote deployments without external supervision.
PART II: NETWORKING & WEB PROTOCOLS (DAYS 4-5)		
Day 4	Networking: OSI & Data Flow - OSI Model vs. TCP/IP stack - TCP 3-way handshake & termination - UDP vs. TCP protocol trade-offs - IP addressing, Subnetting & CIDR - DNS resolution workflow	Builds a robust mental model of data transmission, which is critical for analyzing API latencies, network configurations, and packet loss.

Day 5	Web Networking & App Layer Protocols ▪ HTTP/HTTPS methods, headers, status codes ▪ SSL/TLS secure handshake ▪ RESTful API design constraints ▪ WebSockets vs. SSE (Server-Sent Events)	Provides an understanding of secure, stateless web communication, allowing interns to build standardized REST and real-time APIs.
PART III: DATABASE MANAGEMENT SYSTEMS & STORAGE (DAYS 6-7)		
Day 6	Relational Databases (RDBMS) & SQL ▪ ACID transactional guarantees ▪ SQL Joins and Subqueries ▪ Database Indexing (B-Trees) ▪ Query Execution Plan analysis ▪ PostgreSQL Architecture (MVCC)	Establishes database transactional consistency. Interns learn to design robust tables and optimize queries to avoid production-halting database locks.
Day 7	NoSQL Databases & Distributed Data ▪ CAP Theorem trade-offs ▪ Document Databases (MongoDB) ▪ Key-Value caching (Redis) ▪ Wide-Column stores (ScyllaDB / Cassandra) ▪ Eventual vs. Strong Consistency	Equips interns with the knowledge of when to swap traditional ACID guarantees for horizontal scale, high throughput, and fault tolerance.
PART IV: CLOUD ARCHITECTURE & DEVOPS LIFECYCLE (DAYS 8-12)		
Day 8	Cloud Computing: Core Infrastructure ▪ Virtualization vs. Bare Metal ▪ Cloud service models (IaaS, PaaS, SaaS) ▪ AWS/GCP core services (EC2/GCE, S3) ▪ Elasticity & cloud billing models	Shifts their developer mindset from running scripts on localhost to deploying elastic, highly available infrastructure across global server networks.
Day 9	Cloud Security & Isolation Systems ▪ Virtual Private Cloud (VPC) ▪ Subnets, Gateways & Route Tables ▪ Security Groups & Network ACLs ▪ IAM Roles & Principle of Least	Teaches security-first cloud architecture. Interns learn how to isolate networks and lock down resource permissions before deployment.

	Privilege	
Day 10	DevOps: Containerization with Docker ▪ Containers vs. Virtual Machines ▪ Docker Engine & architecture ▪ Writing optimized Dockerfiles ▪ Docker Networking & Volume mounts ▪ Multi-container setups (Docker Compose)	Eliminates the classic 'works on my machine' bug, packaging code and all runtime dependencies into self-contained, predictable environments.
Day 11	Container Orchestration with Kubernetes ▪ Control Plane vs. Worker Nodes ▪ Pods, Services, and Deployments ▪ Declarative config via YAML ▪ Self-healing & Horizontal Autoscaling	Exposes interns to how major companies manage thousands of microservices dynamically, introducing auto-healing and zero-downtime rolling updates.
Day 12	Infrastructure as Code & Observability ▪ Terraform declarative syntax & states ▪ Monitoring vs. Observability ▪ Metrics collection (Prometheus) ▪ Visualizing logs & states (Grafana) ▪ Defining actionable alert rules	Enforces infrastructure-as-code discipline, while teaching interns how to track resource metrics and catch application issues before users report them.
PART V: DATA STRUCTURES & ALGORITHM DESIGN PATTERNS (DAYS 13-14)		
Day 13	DSA Foundations: Time-Space Complexity ▪ Big-O notation & growth rates ▪ Memory layout of data structures ▪ Arrays vs. Linked Lists ▪ Cache locality first principles ▪ Stack vs. Heap allocations	Demystifies how computer memory handles data structures at the hardware level, training interns to write memory-efficient code from day one.
Day 14	Advanced Data Structures & Indexing ▪ Hash Tables & collision resolution ▪ Trees (BST, Trie structures) ▪ Graph traversal (DFS & BFS) ▪ Recursion vs. Iteration limits	Explains the mechanics of constant-time $O(1)$ lookups and details the algorithms underlying search indexes, database lookups, and routing tables.

PART VI: SYSTEM DESIGN & MASSIVE SCALING PRINCIPLES (DAYS 15-17)

Day 15	System Design: Scaling Foundations ▪ Horizontal vs. Vertical scaling ▪ Load Balancer algorithms (L4 vs. L7) ▪ Caching strategies (Write-Through, Eviction) ▪ Content Delivery Networks (CDNs)	Teaches the principles of scaling, enabling interns to design services that withstand heavy user traffic spikes without system crashes.
Day 16	System Design: Decoupled Systems ▪ Monoliths to Microservices ▪ API Gateway routing & rate limiting ▪ Message Queues (RabbitMQ) ▪ Event Streaming (Apache Kafka) ▪ Sync vs. Async processing pattern	Teaches how to design distributed microservices, ensuring that a slow component or background database lock doesn't take down the entire system.
Day 17	System Design: Fault Tolerance & HA ▪ Database replication topologies ▪ Database Sharding & Partitioning ▪ Circuit Breaker pattern ▪ Graceful degradation & failover	Provides interns with the architectural tools to ensure software remains highly available and resilient during system outages or partial failures.

PART VII: REAL-WORLD ARCHITECTURAL CASE STUDIES (DAYS 18-21)

Day 18	Design Thinking & Solutions Architecture ▪ Analyzing customer problem statements ▪ Translating product requirements to tech specs ▪ Drafting RFCs & System Design documents ▪ Architectural cost vs. benefit analysis	Expands their focus from raw code to product engineering, training interns to understand business goals and draft clean technical blueprints.
Day 19	Case Study 1: TBD Product For Example Uber (Geospatial Dispatch) ▪ Uber's dispatch system architecture ▪ Geospatial indexing with H3 Hexagons	Applies theoretical data structures and networking to a global scale business case, demonstrating how spatial geometry indexes solve ridesharing dispatch.

	▪ Real-time tracking via WebSockets/gRPC ▪ Matchmaking lock mechanics	
Day 20	Product Case Study 2: TBD	
Day 21	Product Case Study 3: TBD	
PART IX: INTERNET OF THINGS (IOT) & EDGE COMPUTING (DAYS 22-23)		
Day 22	IoT & Embedded Systems Foundations ▪ Microcontrollers (ESP32) vs. SBCs (Raspberry Pi) ▪ Analog vs. Digital signals ▪ GPIO programming first principles ▪ Sensor integration and serial protocols	Connects software logic to the physical world, teaching interns how computers interface with physical sensors and environment controllers.
Day 23	IoT Protocol Stack & Edge Ingestion ▪ Publish-Subscribe pattern ▪ MQTT protocol vs. HTTP/CoAP ▪ Edge gateways & local pre-processing ▪ Time-series database ingestion	Teaches lightweight network protocols. Interns understand how to optimize communication in resource-constrained or low-bandwidth environments.
PART X: GENERATIVE AI & AGENTIC WORKFLOW AUTOMATION (DAYS 24-27)		
Day 24	GenAI: LLM Developer Fundamentals ▪ LLM architectures (Transformers) ▪ Tokenization, context windows, & cost ▪ Prompt engineering design patterns ▪ Low-code automation with n8n	Teaches interns to build functional AI solutions by programmatically linking LLMs with external APIs and automated business workflows.
Day 25	AI Agents & Local LLM Deployments ▪ Agentic autonomy & ReAct pattern ▪ Semantic Search & Embeddings ▪ Vector Databases (Chroma/pgvector)	Introduces the latest frontier in software engineering: building autonomous, local AI agents capable of choosing actions and tools.

	▪ Function calling & external tool bindings ▪ Agent runtime security & execution sandboxing	
Day 26	Code Agents & Self-Debugging Loops • OpenCode	Exposes interns to iterative coding agents, demonstrating how to use AI pairs to write, run, and self-correct code from errors.
Day 27	Advanced Agent Systems ▪ Multi-agent orchestration (OpenClaw, Hermes)	Explore on utilizing Advanced copilot AI agents through OpenClaw , Hermes
PART XI: OPEN SOURCE CONTRIBUTION		
Day 28	Open Source & GSOC Counseling ▪ Understanding Open Source licensing ▪ Finding 'good first issues' on GitHub ▪ Google Summer of Code (GSoC) roadmap ▪ Building a public technical brand	Encourages community contribution, giving interns a blueprint for building a strong, globally recognized portfolio using open-source projects.

Weekly Learning Outcomes & Milestones

- Week 1 (OS, Linux, Networking & Databases - Days 1–7): Interns master CLI command-line navigation and script automation. They exit this week understanding OS process execution, standard network transmission models, and relational vs. non-relational database design concepts.
- Week 2 (Cloud, DevOps, IaC & DSA - Days 8–14): Interns shift to cloud environments, package software using Docker, manage containers in Kubernetes, structure infrastructure with Terraform, and deep-dive into time-space complexity metrics and algorithm structures.
- Week 3 (System Design, Case Studies & Solutions Design - Days 15–21): Interns design scalable, decoupled architectures using load balancers and asynchronous messaging queues. They analyze massive architectural studies (Uber, Discord, Notion) and author professional RFC design documents.
- Week 4 (IoT, AI Agents & Career Launch - Days 22–30): Interns bridge hardware-software boundaries, deploy local LLM instances to run autonomous agentic execution loops.